图论算法讲义

最短路径 · 最小生成树

Dijkstra / Floyd / Kruskal / Prim

概念讲解 → 手算步骤 → 截图练习 → 详细解答

适用：数据结构与图论入门；重点训练读图、填表、写路径与判断成环。

这份讲义怎样使用

每个算法先读概念和操作步骤，再独立完成练习，最后核对解答。练习页与答案页分开，建议暂时遮住后面的答案。所有距离表均明确记录时机，避免把“本轮选点”与“本轮更新”混在一起。

内容	包含的学习任务
基础知识	读懂有向图、无向图、边权、路径、∞ 与邻接矩阵
Dijkstra	从一个起点出发；确定顶点、松弛、前驱与路径恢复
Floyd	求全部点对；逐个引入中间点；矩阵迭代与截图勘误
最小生成树	理解连通、无环、n−1 条边与总权值最小
Kruskal	按边权排序，跳过成环边，记录连通分量
Prim	从顶点扩展，维护跨越已选集合的最小连接边
复习与自测	算法选择、复杂度、易错点与自测答案

截图的使用说明

截图 1、2 用于 Dijkstra；截图 3、4 用于 Floyd。截图 4 按截图 3 的图与矩阵核算后存在两处结果差异，正文保留原图并给出更正。最小生成树练习由截图 1 去掉箭头改编，明确标注为补充题，不将有向图直接当作普通最小生成树题。

术语：Dijkstra（迪杰斯特拉）、Floyd（弗洛伊德）、Kruskal（克鲁斯卡尔）、Prim（普里姆）。

01 基础：先分清“最短路”和“最小生成树”

1. 图与边权

图由顶点和边组成。边上的数字是权值，可以表示距离、时间或费用。有向图必须顺着箭头走；无向图的一条边可以双向使用。a→b 存在，不意味着 b→a 也存在。两条线相交但没有标出顶点时，不能把交点当作新顶点。

2. 路径与路径长度

路径是一串相邻顶点，路径长度是沿途各边权值的和。最短路找的是权值和最小的路径，而不一定是边数最少的路径。

例如 s→t 的直接边权为 10，而 s→u→t 的两条边权为 2、3，则最短距离是 5。

3. ∞、0 与邻接矩阵

矩阵行表示起点、列表示终点。W[i][j] 是 i→j 的直接边权；没有直接边时记 ∞。∞ 表示当前不存在已知路径，不表示 0。用于标准最短路算法的距离矩阵中，D[i][i]=0，因为可以留在原点、不经过任何边。

问题	最短路径	最小生成树
优化目标	某个起点到终点的路程最小	连接所有顶点的边权总和最小
结果形态	一条路径 / 距离表 / 距离矩阵	一棵包含所有顶点的树
典型算法	Dijkstra、Floyd	Kruskal、Prim
是否指定起点	Dijkstra 需要；Floyd 求全部点对	Prim 可选起点，但优化目标不变
常规适用图	有向或无向带权图	连通、无向带权图
负边权	Dijkstra 不允许；Floyd 有条件允许	允许负权；仍按大小比较

最短路径树不一定是最小生成树。一个最小化“从起点走到各点的距离”，另一个最小化“选出的全部边的总成本”。

02 Dijkstra：概念与手算方法

1. 核心想法

从起点出发，先记录目前能找到的最短距离。每轮从尚未确定的顶点中，选暂定距离最小者，把它的距离确定下来，再检查经过它能否使其他顶点更近。

记号	含义
s	源点 / 起点
S	最短距离已经确定的顶点集合
dist[v]	从 s 到 v 的当前最好距离
prev[v]	当前路径中 v 的前一个顶点

2. 一次“松弛”到底比较什么

dist[v] ← min(dist[v], dist[u] + w(u,v))

左边是原来从 s 到 v 的最好距离；右边是先到 u、再走 u→v 的距离。只有右边更小时，才更新 dist[v]，并把 prev[v] 改成 u。不能只比较 w(u,v) 的大小。

3. 手算步骤

① 初始化：dist[s]=0，其余为 ∞；S 为空。
② 选点：在 V−S 中选择 dist 最小的 u。
③ 确定：把 u 加入 S。
④ 松弛：沿 u 的出边更新尚未确定的顶点。
⑤ 重复：所有可达点确定后结束；剩余最小值为 ∞ 时也可停止。

```
dist[s] = 0，其他 dist = ∞
while 存在未确定顶点：
    u = 未确定顶点中 dist 最小者
    if dist[u] == ∞：结束
    将 u 加入 S
    for 每条 u→v（v 未确定）：
        if dist[u] + w(u,v) < dist[v]：
            dist[v] = dist[u] + w(u,v)
            prev[v] = u
```

4. 为什么必须要求非负边权

非负权保证：绕到一个尚未确定、距离更大的顶点之后，不可能再靠负边把已确定点的距离拉低。这是每轮确定最小暂定距离的依据。标准 Dijkstra 可处理 0 权边，但不能用于含负权边的图。

02 Dijkstra：截图练习

练习 A：对下图，以 a 为源点，求 a 到其余各点的最短路径和最短距离。先独立填写表格，下一页再核对。

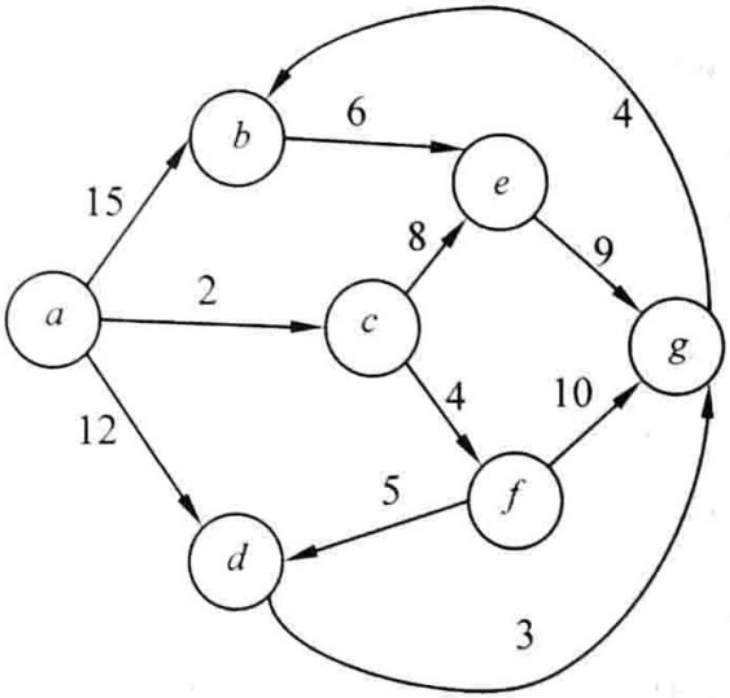


图 4-30　带权的有向图

原始截图 1：图 4-30，带权有向图。特别注意 f→d、d→g、g→b 的方向。

要求：①写确定顶点的顺序；②逐轮更新距离；③写各终点的路径；④解释为什么 a→g 的最短路不是 a→c→f→g。

选定顶点	S	b	c	d	e	f	g
初始化	∅	∞	∞	∞	∞	∞	∞
____	____	____	____	____	____	____	____
____	____	____	____	____	____	____	____
____	____	____	____	____	____	____	____
____	____	____	____	____	____	____	____
____	____	____	____	____	____	____	____
____	____	____	____	____	____	____	____
____	____	____	____	____	____	____	____

本练习统一采用“选定本轮顶点，并完成本轮松弛后”的记录方式；最后一轮即使没有更新，也记录一行。

02 Dijkstra：逐轮解答

表中加 * 的距离已确定；各行均为完成本轮松弛后的状态。a 的距离为 0。

本轮确定	确定集合 S	b	c	d	e	f	g
a	{a}	15	2	12	∞	∞	∞
c	{a,c}	15	2*	12	10	6	∞
f	{a,c,f}	15	2*	11	10	6*	16
e	{a,c,f,e}	15	2*	11	10*	6*	16
d	{a,c,f,e,d}	15	2*	11*	10*	6*	14
g	{a,c,f,e,d,g}	15	2*	11*	10*	6*	14*
b	全部顶点	15*	2*	11*	10*	6*	14*

逐轮解释

① 确定 a：直接到 b、c、d，距离分别为 15、2、12。

② 确定 c（2）：到 e 可得 2+8=10；到 f 可得 2+4=6。下一轮选 f，而不是直接边权更小的其他边。

③ 确定 f（6）：到 d 得 6+5=11，比 12 小；到 g 得 6+10=16。当前最小未确定距离是 e 的 10。

④ 确定 e（10）：经 e 到 g 为 10+9=19，不优于 16，保持不变。

⑤ 确定 d（11）：经 d 到 g 为 11+3=14，比 16 小，更新 g，前驱改为 d。

⑥ 确定 g（14）：经 g 到 b 为 14+4=18，不优于直接到 b 的 15。

⑦ 确定 b（15）：结束。b→e 不会改善已经确定的 e。

完整确定顺序：a → c → f → e → d → g → b。

恢复路径

以 g 为例：prev[g]=d，prev[d]=f，prev[f]=c，prev[c]=a。先逆向得到 g←d←f←c←a，再反过来写 a→c→f→d→g。

02 Dijkstra：最终答案与原表的读法

终点	最短距离	最短路径	边权求和
b	15	a→b	15
c	2	a→c	2
d	11	a→c→f→d	2+4+5
e	10	a→c→e	2+8
f	6	a→c→f	2+4
g	14	a→c→f→d→g	2+4+5+3

表 4-3　从顶点 a 到其他各点的最短路径的求解过程

终点 / Dist	b	c	d	e	f	g	S (终点集)
$i=1$	15 (a,b)	2 (a,c)	12 (a,d)				$\{a,c\}$
$i=2$	15 (a,b)		12 (a,d)	10 (a,c,e)	6 (a,c,f)		$\{a,c,f\}$
$i=3$	15 (a,b)		11 (a,c,f,d)	10 (a,c,e)		16 (a,c,f,g)	$\{a,c,f,e\}$
$i=4$	15 (a,b)		11 (a,c,f,d)			16 (a,c,f,g)	$\{a,c,f,e,d\}$
$i=5$	15 (a,b)					14 (a,c,f,d,g)	$\{a,c,f,e,d,g\}$
$i=6$	15 (a,b)						$\{a,c,f,e,d,g,b\}$

原始截图 2：表 4-3。用于核对结果，不应把空白格理解为距离 0。

为什么截图的 S 与距离变化像“错开了一轮”

原表第 1 行展示由 a 得到的初始候选距离，然后把 c 放入 S；第 2 行才展示经过 c 更新后的 e、f，并将 f 放入 S。也就是说，行内 Dist 主要体现上一已确定顶点的松弛结果，而 S 已包含这行新选出的顶点。

本讲义将“本轮选点＋本轮松弛”写在同一行，因此中间行与截图的编号不直接一一对应，但最终距离和选点顺序一致。原表对已处理列留空是简写；讲义保留全部距离，更适合练习核对。

03 Floyd：概念与矩阵更新

1. 一次解决所有起点到所有终点

Dijkstra 固定一个源点；Floyd 逐个尝试中间顶点，一次求出所有点对的最短距离。它的核心问题是：“如果允许绕经顶点 k，i 到 j 能不能更短？”

D[i][j] ← min(D[i][j], D[i][k] + D[k][j])

例如 i→j 原来为 20，i→k 为 6，k→j 为 8，则经 k 得到 14，于是更新为 14。若某段为 ∞，两段拼不出路径，就不更新。

2. 初始化与轮次含义

初始化矩阵 D 初：直接边写权值，不存在的边写 ∞，对角线写 0。若存在多条同向平行边，先取最小边权。本讲义的例题无自环。

处理 k=0 后，允许顶点 0 作为中间点；处理 k=1 后，允许 0、1 作为中间点；依次扩大允许集合。端点不受这个集合限制，只限制内部中间点。

递推理解：当前阶段的最短路，要么不经过新加入的 k，要么由 i→k 和 k→j 两段拼成。比较这两种可能，就得到新的最短距离。

3. 三层循环的顺序

```
D = 初始化的距离矩阵
for k = 0 ... n-1:          // 中间点，必须放最外层
    for i = 0 ... n-1:      // 起点
        for j = 0 ... n-1:  // 终点
            if D[i][k] 与 D[k][j] 都不是 ∞:
                D[i][j] = min(D[i][j], D[i][k]+D[k][j])
```

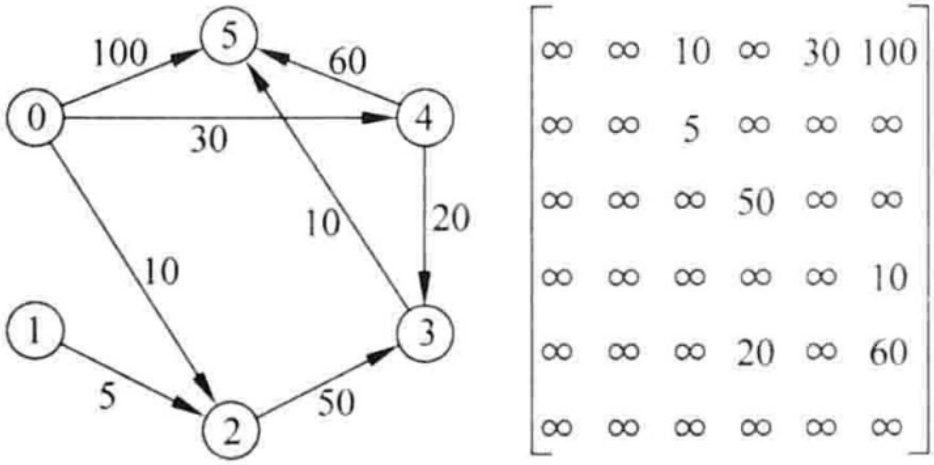
4. 路径记录与负权说明

若要输出路径，可记录 next[i][j]：从 i 去 j 的最短路第一步走向哪里。直接边 i→j 初始化为 j；若经 k 变短，就令 next[i][j]=next[i][k]，之后逐步前进到 j。

Floyd 可以处理负权边，但若某个点对可经由负环反复绕行，距离可以无限下降，不存在有限最短路。标准初始化后若最终 D[v][v]<0，可检测到负环。无向图的一条负权边可往返形成负环，尤其要注意。

03 Floyd：截图练习

练习 B：根据截图 3 的有向图，按中间点顺序 0、1、2、3、4、5 执行 Floyd。下一页开始给出分阶段解答。



原始截图 3：左侧是有向图，右侧是直接边权矩阵。行列顺序均为 0～5。

先检查你读出的边

边	权值	边	权值
0→2	10	0→4	30
0→5	100	1→2	5
2→3	50	3→5	10
4→3	20	4→5	60

截图右侧把对角线写成 ∞，用于表示“没有直接自环”。标准最短距离矩阵初始化时，把对角线改为 0；这不改变本题不同顶点之间的最短距离。

独立练习要求

①写标准初始化矩阵；②找出 k=2、3、4 各轮更新的元素；③写最终距离矩阵；④写 0→5、1→5、2→5 和 4→5 的最短路径；⑤解释为什么 0→2→3→5 不是 0→5 的最短路。

起点\终点	0	1	2	3	4	5
0	____	____	____	____	____	____
1	____	____	____	____	____	____
2	____	____	____	____	____	____
3	____	____	____	____	____	____
4	____	____	____	____	____	____
5	____	____	____	____	____	____

提示：每一轮都要检查全部 i、j。特别留意 2→3→5，以及 0→4→3→5。

03 Floyd：初始化与 k=0、1、2

标准初始化矩阵 D 初

起点\终点	0	1	2	3	4	5
0	0	∞	10	∞	30	100
1	∞	0	5	∞	∞	∞
2	∞	∞	0	50	∞	∞
3	∞	∞	∞	0	∞	10
4	∞	∞	∞	20	0	60
5	∞	∞	∞	∞	∞	0

先处理 k=0、k=1：两个阶段均没有任何元素改善。因此“处理 0 后”和“处理 1 后”的矩阵仍等于 D 初。它们没有来自其他顶点的入边，不能作为内部桥梁提供新的连接。

处理 k=2 后：允许中间点 {0,1,2}

起点\终点	0	1	2	3	4	5
0	0	∞	10	60	30	100
1	∞	0	5	55	∞	∞
2	∞	∞	0	50	∞	∞
3	∞	∞	∞	0	∞	10
4	∞	∞	∞	20	0	60
5	∞	∞	∞	∞	∞	0

这一轮更新的两个元素

元素	旧距离	经 2 计算	新距离 / 对应路径
D[0][3]	∞	D[0][2]+D[2][3]=10+50	60；0→2→3
D[1][3]	∞	D[1][2]+D[2][3]=5+50	55；1→2→3

此时不能把刚出现的 0→3 再接 3→5，当作“经过 2”阶段的结果：路径内部还使用了顶点 3，应在下一轮 k=3 时处理。

按阶段理解：处理 k=2 时，允许的内部顶点只有 0、1、2；顶点 3 可以是终点，但此时不能当内部桥梁。

03 Floyd：k=3、4、5 与最终矩阵

处理 k=3 后：允许中间点 {0,1,2,3}

起点\终点	0	1	2	3	4	5
0	0	∞	10	60	30	70
1	∞	0	5	55	∞	65
2	∞	∞	0	50	∞	60
3	∞	∞	∞	0	∞	10
4	∞	∞	∞	20	0	30
5	∞	∞	∞	∞	∞	0

元素	经 3 计算	更新
D[0][5]	60+10=70	100 → 70
D[1][5]	55+10=65	∞ → 65
D[2][5]	50+10=60	∞ → 60
D[4][5]	20+10=30	60 → 30

处理 k=4 后：允许中间点 {0,1,2,3,4}

起点\终点	0	1	2	3	4	5
0	0	∞	10	50	30	60
1	∞	0	5	55	∞	65
2	∞	∞	0	50	∞	60
3	∞	∞	∞	0	∞	10
4	∞	∞	∞	20	0	30
5	∞	∞	∞	∞	∞	0

本轮两处改善：D[0][3]=min(60,30+20)=50；D[0][5]=min(70,30+30)=60。后一个 30 是上一轮已经求出的 4→3→5 的最短距离。

处理 k=5 后不再变化，因为 5 没有出边。所以上面的矩阵就是最终标准最短距离矩阵。

结果检查：0→5=60，1→5=65，2→5=60，4→5=30。∞ 表示最终仍不可达。

03 Floyd：路径答案与截图勘误

点对	最短距离	最短路径	求和
0→2	10	0→2	10
0→3	50	0→4→3	30+20
0→4	30	0→4	30
0→5	60	0→4→3→5	30+20+10
1→2	5	1→2	5
1→3	55	1→2→3	5+50
1→5	65	1→2→3→5	5+50+10
2→3	50	2→3	50
2→5	60	2→3→5	50+10
3→5	10	3→5	10
4→3	20	4→3	20
4→5	30	4→3→5	20+10

以上列出全部可达的不同顶点点对；其余不同点对为 ∞。各点到自身按标准约定均为 0。

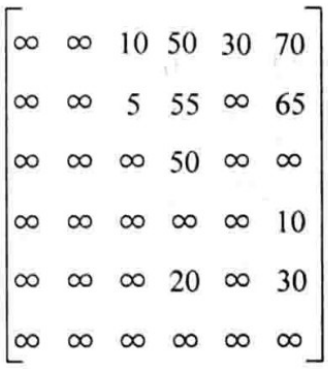


图 4-32 Floyd 最短路径矩阵

原始截图 4：图 4-32，保留原图供对照。

位置	截图 4	按截图 3 应为	依据
第 0 行第 5 列	70	60	0→4→3→5，30+20+10
第 2 行第 5 列	∞	60	2→3→5，50+10

对角线的 ∞ 属于原图表示约定，讲义标准矩阵用 0。上表两处差异涉及不同顶点的距离，不能用对角线约定解释。若沿用截图的“无自环记 ∞”格式，仍应将这两处改为 60。

03 最短路径：手算模板与常见错误

1. Dijkstra 的答题模板

“设源点为 s，已确定集合为 S。初始化距离；每轮选取未确定点中 dist 最小者，加入 S，再松弛其出边。得到确定顺序……；最终距离为……；利用前驱恢复路径为……。”

表格一定写清楚是“松弛前”还是“松弛后”。若题目已有表格格式，先弄清它的记录方式，再填表，不能把两种方式混用。

2. Floyd 的答题模板

“按顶点顺序初始化 D。依次以 k 为中间点，执行 D[i][j]=min(D[i][j],D[i][k]+D[k][j])。记录每轮发生变化的元素，最后给出矩阵和题目要求的路径。”

常见错误	正确做法
把箭头反过来走	只沿出边走；无向图才允许双向
只挑边权最小的边	Dijkstra 挑的是从源点累计的暂定距离
写了距离却没有路径	记录前驱，并从终点倒推到源点
Floyd 只试一个中间点	依次允许所有中间点；每轮检查所有点对
看到图上交叉就能换线	只有明确的顶点才能连接、换向
把 ∞ 算成大数且无条件相加	程序中需判断可达，避免溢出或伪更新
认为最短路就是边数最少	加权最短路比较的是权值和

3. 两个小检查

① 截图 1 中，经 e 到 g 为 19，经 f 直接到 g 为 16，但最短距离为 14；说明“先找到的路径”仍可能被后续更新。

② 截图 3 中，0→5 在 k=3 后为 70，在 k=4 后为 60；说明 Floyd 必须完成所有中间点阶段。

04 最小生成树：概念与判断

1. 什么叫生成树

对一个连通无向图，选出一部分原图中的边，要求包含所有顶点、仍然连通、没有环，这些边构成一棵生成树。若有 n 个顶点，则生成树恰好有 n−1 条边。

2. 什么叫最小生成树（MST）

在所有生成树中，边权总和最小的一棵称为最小生成树。它不要求从某个起点到各点的路线都最短，也不要求选出全图最小的 n−1 条边——那些边可能成环，或不能覆盖所有顶点。

三项验收：覆盖所有顶点；连通且无环；总权值最小。只数到 n−1 条边还不够，还要检查连通与无环。

3. 为什么可以贪心选边

把顶点分成两组，连接两组的边叫跨越这个划分的边。割性质告诉我们：跨越一个划分的最小权边，在合适的已有选择基础上可安全加入某棵最小生成树。Kruskal 和 Prim 都是在找这样的安全边。

环性质提供另一个视角：一个环中严格比其他边都重的那条边不会出现在任何最小生成树中。若最大边并列，不能武断说某一条一定不在最小生成树中。

4. 相同权值与不连通图

出现相同权值时，可按固定规则打破并列；不同选择可能产生不同的最小生成树，但最优总权值相同。仅有重复权值并不能断言最小生成树一定不唯一。若所有边权互不相同，则最小生成树唯一。

如果图不连通，不存在覆盖全部顶点的一棵生成树。可以对各连通分量分别求树，得到最小生成森林。

算法	从哪里开始	每轮挑什么	怎样避免成环
Kruskal	全图所有边	当前最小、且不成环的边	检查两个端点是否已连通
Prim	任意一个顶点	已选集合与未选集合之间的最小边	每次只引入一个新顶点

普通 MST 针对无向图；有向图的最小树形图是另一个问题，不可直接照搬这两种算法。

05 Kruskal：按边权排序选边

1. 核心想法

把全部边按权值从小到大排列。依次查看每条边：如果加入后不成环，就选入；如果会成环，就跳过。选够 n−1 条边即可停止。

2. 判断成环的等价方法

只检查已经选出的边。如果某条候选边的两个端点在已有森林中已经连通，加入这条边就会成环；如果分属两个连通分量，就可以连接它们。

例如已选 a—c、c—f、f—d，那么 a 与 d 已经通过 a—c—f—d 连通。此时再选 a—d，就形成 a—c—f—d—a 这个环，即使 a—d 本身看起来很短也不能加入。

3. 手算步骤

① 列出所有边与权值，按权值升序排列。
② 初始每个顶点是一个独立连通分量。
③ 看当前边的两个端点是否属于同一分量。
④ 不同分量：选边并合并；同一分量：跳过。
⑤ 选够 n−1 条边后停止，求总权值。

```
将全部边按权值升序排序
每个顶点建立一个独立集合
T = 空集
for 每条边 (u,v,w):
    if find(u) != find(v):
        将边加入 T
        union(u,v)
    if T 有 n-1 条边：结束
```

4. 并查集的直观解释

find(u) 用来找 u 所属分量的代表；union(u,v) 用来合并两个分量。手算时不必画复杂的父指针结构，直接写 {a,c}、{d,g} 这样的集合就能判断。

Kruskal 按“全图的边权”排序；它不会维护从某个起点出发的累计距离。已选边在中途可以暂时组成多棵小树。

06 Prim：从一棵树向外扩展

1. 核心想法

先任选一个顶点作为已选集合 U。每轮只考虑一端在 U、另一端在 V−U 的边，挑最小权的一条，将新顶点和该边加入树。重复到所有顶点加入。

Prim 每次比较的是“连接已有树的一条边的权值”，不是“从起点走到这个顶点的累计距离”。

2. key 与 parent 的含义

key[v] 表示未加入树的 v 与当前 U 之间最轻连接边的权值；parent[v] 是这条边在 U 中的端点。如果 v 暂时没有边连到 U，key[v]=∞。

新顶点 u 加入后，对于尚未加入的邻点 v，检查 w(u,v)<key[v]：若成立，更新 key[v]=w(u,v)、parent[v]=u。这里绝不能加上 key[u]。

操作	Dijkstra	Prim
选点依据	最小 dist[v]	最小 key[v]
更新公式	dist[v] ← min(dist[v],dist[u]+w(u,v))	key[v] ← min(key[v],w(u,v))
表的含义	从源点到 v 的累计距离	v 到当前树的最轻连接边
结果	单源最短路径	最小生成树

3. 手算步骤与伪代码

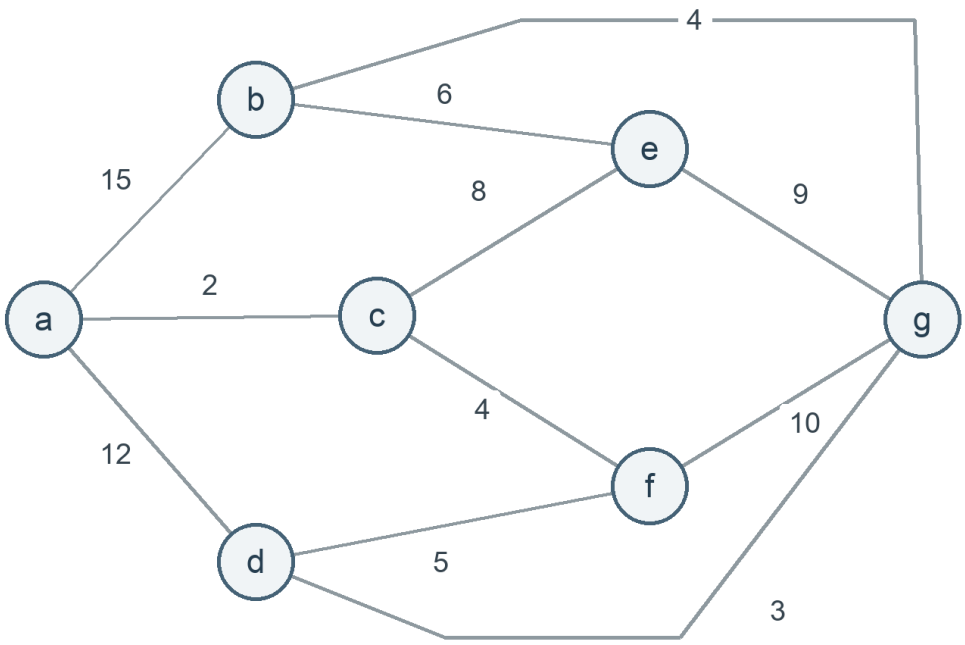
选起点；把起点放入 U；更新与它相连的 key。以后每轮挑最小 key 的未选点，加入对应 parent 边，再更新剩余点的 key。

```
key[start] = 0，其他 key = ∞
U = 空集
while 仍有未加入顶点：
    u = 未加入点中 key 最小者
    if key[u] == ∞：图不连通，无法继续得到整图生成树
    将 u 加入 U
    若 u 不是起点，将 (parent[u],u) 加入树
    for 每个未加入的邻点 v：
        if w(u,v) < key[v]：
            key[v] = w(u,v)；parent[v] = u
```

改变起点可能改变选边顺序；有并列时，也可能得到不同的最小生成树。但只要执行正确，最优总权值不变。

04—06 最小生成树：由截图 1 改编的练习

练习 C：将截图 1 中的所有箭头去掉，保留相同的顶点、边与权值，得到下面的无向图。它是补充改编题，原截图 1 本身仍是有向最短路径题。



改编规则：原 g→b 变为 b—g，原 f→d 变为 d—f，其余边同样只去掉方向。

边	权值	边	权值	边	权值
a—b	15	a—c	2	a—d	12
b—e	6	c—e	8	c—f	4
d—f	5	f—g	10	e—g	9
d—g	3	b—g	4	—	—

练习要求

① 用 Kruskal 求最小生成树，列边的升序、取舍和理由。
② 用 Prim 从 a 开始求最小生成树，逐轮写 U、选边及 key 更新。
③ 写出最小生成树的总权值。
④ 对比两算法的选边顺序，并说明结果是否一致。

为了便于核对，本题 Kruskal 的两条权值 4 的边固定按 b—g 在前、c—f 在后处理；交换这两条边的次序也可以。

Kruskal：选边顺序 _____；总权值 ____。
Prim（从 a 开始）：选边顺序 _____；总权值 ____。

05 Kruskal：练习 C 的详细解答

1. 全部边按权值排序

序号	边	权值	处理
1	a—c	2	选入
2	d—g	3	选入
3	b—g	4	选入
4	c—f	4	选入
5	d—f	5	选入
6	b—e	6	选入，已达到 6 条边，停止
7	c—e	8	无需继续处理
8	e—g	9	无需继续处理
9	f—g	10	无需继续处理
10	a—d	12	无需继续处理
11	a—b	15	无需继续处理

2. 连通分量怎样变化

选边	合并后的连通分量
初始	{a}、{b}、{c}、{d}、{e}、{f}、{g}
a—c	{a,c}、{b}、{d}、{e}、{f}、{g}
d—g	{a,c}、{d,g}、{b}、{e}、{f}
b—g	{a,c}、{b,d,g}、{e}、{f}
c—f	{a,c,f}、{b,d,g}、{e}
d—f	{a,b,c,d,f,g}、{e}
b—e	{a,b,c,d,e,f,g}

MST 边集：{a—c, d—g, b—g, c—f, d—f, b—e}；总权值 2+3+4+4+5+6=24。

本题前 6 条边都连接不同分量，因此真正执行时没有出现跳过边。如果继续检查剩余边，它们的端点均已连通，加入任何一条都会成环。不能把“前 6 条恰好都可选”当作所有题的通用规则。

例如继续看 c—e，会与已有的 c—f—d—g—b—e 构成环，必须舍弃。

06 Prim：练习 C 的详细解答

1. 从 a 开始逐轮扩展

轮次	加入前的 U	选边	边权	加入后
1	{a}	a—c	2	{a,c}
2	{a,c}	c—f	4	{a,c,f}
3	{a,c,f}	f—d	5	{a,c,f,d}
4	{a,c,f,d}	d—g	3	{a,c,f,d,g}
5	{a,c,f,d,g}	g—b	4	{a,c,f,d,g,b}
6	{a,c,f,d,g,b}	b—e	6	全部顶点

2. key 表：完成本行顶点加入与更新后的状态

本行加入	b	c	d	e	f	g
a	15(a)	2(a)	12(a)	∞	∞	∞
c	15(a)	已入	12(a)	8(c)	4(c)	∞
f	15(a)	已入	5(f)	8(c)	已入	10(f)
d	15(a)	已入	已入	8(c)	已入	3(d)
g	4(g)	已入	已入	8(c)	已入	已入
b	已入	已入	已入	6(b)	已入	已入
e	已入	已入	已入	已入	已入	已入

表中 5(f) 表示：当前连接 d 的最轻边是 f—d，权值为 5，parent[d]=f；它不是从 a 到 d 的路径长度。

3. 为什么先选 5、后选 3

在第 3 轮之前，d、g 都在 U 外，d—g 的权值虽为 3，却不是跨越 U 与 V−U 的候选边。必须先通过 f—d 把 d 接入树，下一轮 d—g 才能成为候选，所以 Prim 的选边权值不一定逐轮递增。

Prim 总权值：2+4+5+3+4+6=24，与 Kruskal 相同；本题两者的最终边集也相同。

本题最小生成树唯一，但边权有两个 4。由此也能看出：重复权值仅意味着“可能不唯一”，不能直接推断“一定不唯一”。

07 复习速查与自测答案

算法	目标 / 条件	典型时间复杂度	额外空间（不含图）
Dijkstra	单源；边权非负	朴素 $O(V^2)$；邻接表＋二叉堆 $O((V+E)\log V)$	数组 $O(V)$；允许重复入堆可达 $O(V+E)$
Floyd	所有点对；有限最短路需排除相关负环	$O(V^3)$	距离 / 路径矩阵 $O(V^2)$
Kruskal	无向 MST；不连通则森林	排序主导 $O(E \log E)$	并查集 $O(V)$；另有边表 / 排序开销
Prim	无向 MST；整图需连通	矩阵 $O(V^2)$；邻接表＋二叉堆 $O((V+E)\log V)$	数组 $O(V)$；重复入堆可达 $O(V+E)$

V、E 分别表示顶点数与边数。Dijkstra 的邻接矩阵图存储本身需 $O(V^2)$；Prim 同理。Floyd 的距离矩阵是核心工作空间。复杂度说明按常见实现给出，手算时优先理解公式和选点规则。

四句记忆

Dijkstra：挑最小的累计距离，定点后向外松弛。

Floyd：每次多允许一个中间点，比较经它走会不会更短。

Kruskal：全图边从小到大，连接不同分量就选。

Prim：从已有树往外接，每次挑最轻的跨集合边。

先自测，再看下表

1. 含负权边的有向图能直接用 Dijkstra 吗？
2. Prim 更新 key[v] 时，是否加 key[u]？
3. 7 个顶点的生成树应有多少条边？
4. Kruskal 中两个端点已连通的边能否选？
5. Floyd 截图题中 0→5 为何不是 70？
6. 最小生成树是否总能保证起点到每个点的路最短？

题号	答案
1	不能；标准 Dijkstra 要求非负边权。
2	不加；key 比较单条连接边权，直接比较 w(u,v)。
3	6 条，同时要满足连通且无环。
4	不能；加入后会成环，应跳过。
5	还可经 4：0→4→3→5，总权值 60；70 只是中间阶段结果。
6	不能；MST 优化全部选边的总权值，最短路优化端点间路径。

例题答案速查：Dijkstra 的 b,c,d,e,f,g 距离为 15,2,11,10,6,14；Floyd 的 0→5 与 2→5 均为 60；补充 MST 题总权值为 24。